

Context Mode

Technical architecture, context savings, failure modes and evidence

A research paper for understanding the `mksglu/context-mode` project

Prepared as a foundation for a later technical blog post

Evidence checked against the repository, source code, project benchmarks, community discussion, current issue reports and primary documentation.

Research cut-off: 7 September 2026

Bottom line. Context Mode is a credible and unusually transparent implementation of output-side context externalisation: it executes or indexes large payloads locally, returns a small result, and can retrieve exact fragments later. The repository's 95-100% fixture reductions are plausible for the selected payloads. They are not, by themselves, evidence of equivalent end-to-end token-bill savings, task-quality gains, or universal interception of MCP output. The honest recommendation is measured adoption with an A/B harness, especially for large logs, browser snapshots, documentation and repeated MCP queries.

Contents

1. Executive summary and evidence boundary
 2. The problem: context is working memory, not merely a token bill
 3. What Context Mode is and is not
 4. Architecture in one picture
 5. Technical internals, from hooks to SQLite
 6. Where it helps in practice
 7. Where it can be harmful or disappointing
 8. What the evidence actually shows
 9. Relationship to native Claude controls, RTK and Code Mode
 10. A rigorous evaluation plan for real workflows
 11. Practical adoption guidance
 12. Balanced verdict
- Appendix A. Glossary
- Appendix B. Source register and evidence grading

How to read this paper. The first chapters build the mental model slowly. The middle chapters inspect the implementation and then test the claims against evidence. The final chapters turn the findings into an evaluation protocol. Project-reported numbers are deliberately marked as project evidence; community reports are treated as operational signals, not controlled experiments.

1. Executive summary and evidence boundary

Context Mode is an open-source MCP server and hook system for keeping bulky tool output out of an agent's visible conversation. The central idea is simple: leave the raw payload in a local process or local SQLite database, and return only a compact summary, a pointer, or a small set of retrieved chunks. The project combines this with session-event capture and compaction recovery. [1]

The implementation matters because the phrase context savings can describe several different quantities. A 98% reduction in the bytes returned by one tool is not automatically a 98% reduction in a session's billed input tokens. Nor does it prove that the model will solve the same task just as well. A proper assessment therefore has to track payload size, API usage, cache behaviour, latency, retrieval accuracy, task success and omission errors separately.

The most defensible positive claim is narrow: for large, disposable or queryable outputs, Context Mode can substantially reduce the amount of raw material placed into the next model turn while retaining a local path to the original data. The repository's own fixtures show this clearly. Its benchmark reports 315 KB of executor inputs becoming 5.5 KB of returned context, and 60.3 KB of indexed documentation becoming 11 KB across three searches. [2]

The most important qualification is equally narrow: these are component-level, author-run fixture measurements. I found strong adoption signals, including roughly 20,000 GitHub stars, a large Hacker News discussion and an active issue tracker, but I did not find a neutral, controlled, end-to-end study showing that Context Mode reliably reduces the final Claude Code bill or improves coding-task success across users and platforms. Popularity is evidence of interest and maintenance, not causal evidence of efficacy. [7, 16]

Evidence grading used throughout. Repository fact means visible in current code or documentation. Project benchmark means measured by the maintainers on their fixtures. Community signal means an issue, discussion or independent observation. External evidence means a primary source outside this project. These categories are not interchangeable.

Findings at a glance

Question	Best-supported answer	Confidence
Does it reduce raw output entering context?	Yes, when the agent uses <code>ctx_execute</code> , <code>ctx_execute_file</code> , or <code>index/search</code> , and the task is compatible with compact output or retrieval.	High for mechanism; high for selected fixtures
Does it preserve all information?	Index/search can preserve exact chunks. Summarising and stdout filtering can omit information unless the query is designed well. Large-output auto-indexing helps, but retrieval is still selective.	High
Does it intercept every external MCP result?	No. Current hooks detect external MCP calls and periodically nudge the agent, but the external result remains the external tool's result; the hook does not rewrite that payload.	High
Does it reduce the final token bill?	Plausible in sessions with repeated large tool results, but not established by a neutral end-to-end benchmark. Prompt caching and extra retrieval calls complicate the calculation.	Low to medium
Can it create new failure modes?	Yes: routing false positives, runtime and hook failures, retrieval misses, misleading summaries, overhead on small calls and security exposure from arbitrary local code execution.	High

2. The problem: context is working memory, not merely a token bill

A language-model agent does not see a filesystem or a remote API in the human sense. It sees the text that the host application places into the next request. That request includes system instructions, tool definitions, prior messages, tool results, the new user message and the model's requested output budget. Anthropic describes the context window as working memory and explicitly warns that more context is not automatically better because accuracy and recall can degrade as context grows. [12]

This creates three separate engineering problems. First, a large result consumes space in the current working set and in later turns. Secondly, the model has to find the relevant fact inside the result, which is a retrieval problem rather than a storage problem. Thirdly, repeated turns can accumulate the same noisy material or trigger compaction, truncation or context-window errors. A tool that reduces bytes can help all three, but only if it retains the information needed for the task.

The cost problem is related but not identical. A raw tool result generated outside the model is normally returned as input on a subsequent model request. If Context Mode keeps the raw payload local and sends only a compact result, it can reduce later input tokens. However, the local executor, indexing work and follow-up searches add their own latency and tool calls. If a summary causes the agent to ask several corrective questions, the total session may become larger even though each individual result is small.

Four quantities that should never be conflated

Quantity	Meaning	What Context Mode can change
Raw payload size	Bytes or characters emitted by a command, browser, API or MCP server.	Often unchanged. The raw data may be stored locally instead of shown to the model.
Context consumption	Tokens from tool results and other prompt components that occupy the model's working context.	This is the direct target of output externalisation and on-demand retrieval.
Billed input tokens	Provider accounting for input tokens, often separated into cache reads, cache writes and ordinary input.	May fall when raw results no longer reappear, but it must be measured from usage fields rather than inferred from bytes.
Task success and quality	Whether the agent solves the problem correctly, with acceptable latency and without omissions.	Can improve through less noise, or deteriorate if filtering or retrieval hides a critical detail.

A useful mental model is external memory with a narrow interface. Context Mode does not make the model's window larger. It changes which facts cross the boundary into that window, and when. That makes routing and retrieval quality first-class parts of the agent system.

3. What Context Mode is and is not

The current repository presents Context Mode as a local MCP server with a Claude Code plugin and adapters for several agent environments. Its main tools are `ctx_execute`, `ctx_execute_file`, `ctx_batch_execute`, `ctx_index`, `ctx_search` and `ctx_fetch_and_index`. The first group computes over data and prints a deliberately small result. The second group stores content in a local search index and retrieves relevant exact fragments later. [1]

The project also includes meta-tools such as `ctx_stats`, `ctx_doctor`, `ctx_upgrade`, `ctx_purge` and `ctx_insight`, plus hooks around tool use, prompts, stopping, compaction and session start. These hooks are intended to encourage the agent to choose the compact route and to preserve useful state across a long session. They are part of the behaviour, not merely installation convenience.

It is not a single kind of compression

- It is not primarily an LLM summariser. The default execution path uses code written by the agent and exposes `stdout`; the search path returns stored chunks rather than an LLM-generated paraphrase.

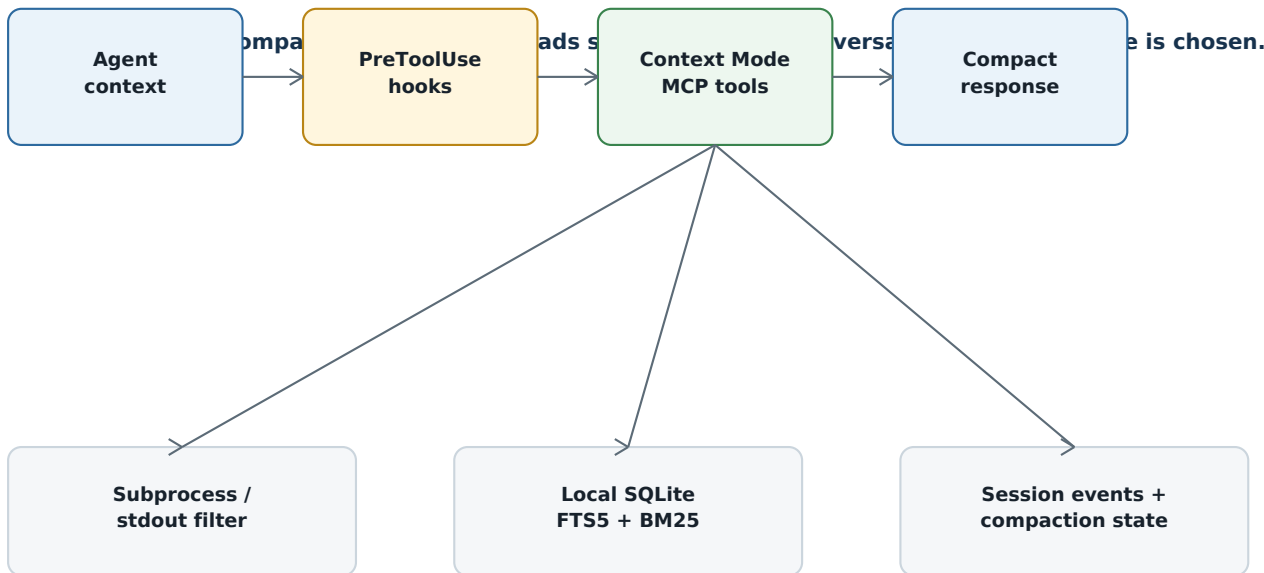
- It is not a universal MCP proxy. External MCP results are not currently rewritten after the external server returns them. The current hook path provides periodic guidance before the call, so success depends on the agent following the guidance.
- It is not a complete operating-system sandbox. The executor uses a child process and output limits, but the repository itself warns that arbitrary code can inherit filesystem access. Host-level sandboxing remains important. [1, 4]
- It is not a guarantee of lower cost. A compact response can protect the context window while adding local compute, extra retrieval calls or a prompt-cache change. Billing must be measured using the provider's usage fields.
- It is not a replacement for careful query design. A model asking for the wrong statistic, an incomplete grep pattern or an overly narrow search query can obtain a compact but wrong answer.

The two main modes of information handling

Mode	What happens	Strength	Main risk
Compute and summarise	The agent asks the executor to parse, count, filter or aggregate. Only stdout, subject to caps and intent handling, returns to the conversation.	Very high reduction for logs, CSV, test output, build output and browser snapshots.	The chosen computation may omit a rare but important detail.
Index and retrieve	The full content is chunked and stored in SQLite FTSS. Later queries return matching snippets or code blocks.	Exact text can be recovered without putting the whole source in context.	Recall depends on chunking, query terms, ranking and the model deciding to search.

4. Architecture in one picture

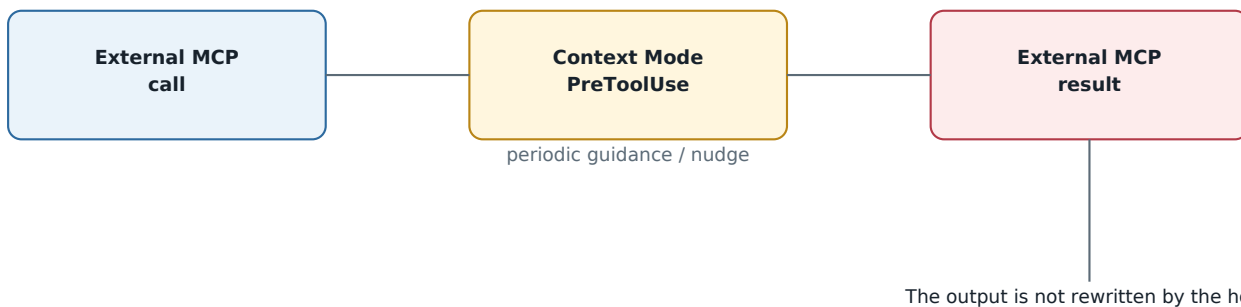
The architecture has a visible path and a hidden local path. The visible path is the small tool result placed into the agent conversation. The local path contains subprocess output, indexed chunks, cache entries and session events. The design is effective when the local path retains what the next reasoning step needs and the visible path communicates how to retrieve it.



Persistent local state is queryable, but retrieval is an additional decision point and tool call.

The hook layer is important because an MCP server alone cannot force an agent to use it. In Claude Code, PreToolUse hooks can deny, modify or add context before a tool call; PostToolUse hooks can record what happened. Context Mode uses that control point to redirect WebFetch, some HTTP commands and verbose build commands, to nudge Read and Grep towards compact routes, and to add guidance for external MCP calls. [5, 6]

External MCP is the critical boundary



The current routing source is unusually explicit about this boundary: external MCP tools are detected, but the branch does not deny or modify the call. It returns periodic guidance, normally every ten matching calls, telling the agent to pipe large results through `ctx_execute`. The external server's response still reaches the host in its original form unless the agent chooses the alternative route. [5]

This means the repository's current implementation partially addresses the earlier community criticism that external MCP responses bypassed Context Mode: it can now notice the call and remind the model. It does not solve the stronger problem of intercepting and transforming the response after the external MCP server has produced it.

5. Technical internals, from hooks to SQLite

This chapter moves from the control plane to the data plane. The control plane decides which route to use. The data plane executes code, limits output, stores content, ranks matches and reconstructs session state. Each layer has a different failure mode, so judging the whole system from a single savings percentage is unsafe.

5.1 Hook routing and redirection

The Claude Code adapter registers matchers for common tools, including Bash, WebFetch, Read, Grep, Agent, Context Mode tools and external MCP namespaces. The core routing module normalises platform-specific names and returns decisions such as passthrough, add context, modify input, deny or ask. [5, 6]

Some routes are deterministic. WebFetch is denied with instructions to use `ctx_fetch_and_index` or `ctx_execute`. A curl or wget call that would emit a body to stdout is redirected towards `ctx_execute`; inline HTTP patterns and certain build tools receive similar treatment. Large Read calls can be annotated when the file exceeds a threshold. These routes prevent a known class of payload floods before the tool runs.

Other routes are advisory. A normal Bash command may receive a one-shot nudge, and external MCP tools receive periodic guidance. The current source comments note that `PreToolUse` runs before the command, so the actual output size is unknowable. The default generic Bash nudge therefore remains broad unless the operator opts into a minimum command-byte threshold. This design favours safety against large output at the cost of potential overhead and false positives.

5.2 Execution: process isolation and stdout as the interface

`ctx_execute` launches a child process for the selected language. The executor captures stdout and stderr, applies timeouts and byte limits, and can terminate the process tree when the cap is exceeded. The conversation receives the resulting text rather than the process's entire intermediate state. The repository supports a dozen runtimes in the current README, with shell commands and authenticated local CLIs inheriting relevant configuration. [1, 4]

This is best understood as output isolation, not as a security boundary. The child process is a useful separation between raw data and model-visible output, but code execution still occurs on the operator's machine. The source includes environment filtering, project-boundary checks for `execute_file` and policy checks for shell commands, yet the documentation explicitly says that this is defence in depth rather than a full OS sandbox. [1, 4]

There is a practical tension here. The more capable the executor is, the more useful it becomes for parsing local files, calling CLIs and fetching data. The same capability makes prompt injection, malicious repository instructions or an incorrect model-generated script more consequential. A production deployment should keep host permissions and tool approvals restrictive, and should make it visually obvious that `ctx_execute` can run arbitrary code.

5.3 Batch execution and progressive throttling

`ctx_batch_execute` groups several commands or searches into one MCP call and can run them concurrently within a bounded range. This reduces repetitive agent-tool round trips when the commands are independent. The repository also uses progressive throttling for repeated external calls: early calls are normal, later calls receive warnings, and sufficiently repeated calls are directed towards batching. [1]

Batching is a performance optimisation, not a semantic guarantee. Concurrent commands can contend for CPU, network or locks, and a failure in one item must be represented clearly enough for the model to distinguish partial success from total failure. A useful benchmark should therefore report per-item success and wall-clock latency, not only one aggregate response size.

5.4 Indexing and retrieval: FTS5, BM25 and exact chunks

The ContentStore uses SQLite with FTS5. Markdown is chunked by headings while retaining code blocks, with a maximum chunk size of roughly 4 KB in the current source. An FTS5 table stores title, content and metadata, and a trigram index supports substring-style matching. The FTS5 tokenizer uses Porter stemming and unicode61. [3]

Ranking is deterministic and local. The SQL query uses BM25 with a heavier weight for titles than content; the source shows a title weighting of 5.0 and a content weighting of 1.0. The broader retrieval pipeline also combines token-based matching with trigram substring matching through Reciprocal Rank Fusion, then applies proximity, typo correction and smart-snippet extraction. This is more sophisticated than a plain grep, but it remains lexical retrieval rather than semantic embedding retrieval. [3, 15]

The trade-off is intentional. Exact retrieval can preserve code examples and signatures that an abstractive summary would distort. However, lexical search can miss a relevant passage when the query uses different terminology, when a structured JSON field is not surfaced as text, or when the relevant fact is an outlier rather than a repeated keyword. A community discussion specifically suggested hybrid semantic retrieval for structured JSON because pure BM25 can underperform there. That is a useful design critique, not a proof that Context Mode fails generally. [7]

5.5 URL fetching, caching and retention

`ctx_fetch_and_index` fetches a page, converts it to Markdown, stores chunks and returns a compact indexing result. The repository documents a local per-project SQLite cache with a default 24-hour TTL. Cache hits can return a small pointer instead of downloading and re-presenting the same page; force or zero-TTL options bypass the cache. [1]

Caching saves repeated transfer and parsing, but it introduces freshness and provenance questions. A cached documentation page may no longer match the live API. A later agent must know the source label, timestamp and query scope. Stale content is especially harmful when a model uses a retrieved code snippet as if it were current. A serious workflow should record the source URL, retrieval time, version and cache policy in the experiment log.

5.6 Session continuity and compaction

The session layer captures events such as decisions, constraints, errors, rejected approaches, active files, external references and tool outcomes into a per-project SQLite database. Before compaction, it reads those events and builds a small priority-tiered XML snapshot. At session start, the snapshot and selected structured events are reintroduced as a session guide. The aim is to preserve state that a generic summary might lose. [1]

This is a second memory system alongside the model context. It can help when a long coding session is interrupted or compacted, but it also creates another source of truth. If the event extractor misclassifies an exploratory idea as a decision, or if an old constraint remains after the code changes, the session guide can bias later reasoning. The appropriate test is not whether a snapshot is small; it is whether recovery preserves the decisions that matter and avoids resurrecting stale ones.

5.7 Security and privacy claims

The repository states that raw data remains local, SQLite is local and there is no telemetry or cloud synchronisation. That is a useful architectural property when the data is sensitive, although local persistence still requires disk-access controls and retention policies. The network fetch layer also includes restrictions for metadata and reserved addresses, plus DNS-rebinding defences, according to the project documentation. [1]

The more material limitation is execution authority. `ctx_execute` can run arbitrary code within the host process environment unless the surrounding agent platform constrains it. `ctx_execute_file` has stronger project-boundary checks, but a symlink, dependency, shell or environment mistake can still matter. The safe interpretation is local processing with policy checks, not an isolated untrusted-code service. [1, 4]

6. Where it helps in practice

Context Mode is most valuable when the raw result is large, repetitive, locally accessible and either aggregatable or searchable. The benefit is not that every byte disappears; it is that the agent does not have to carry every byte in every reasoning turn.

6.1 Large logs, tests and builds

A build log often contains hundreds of successful lines and a small number of errors. A test run may repeat the same stack-trace prefix across many cases. A CSV may contain thousands of rows where the user asks for a count, a distribution or a comparison. In these cases, `ctx_execute_file` can compute the requested statistic and return a small result while leaving the raw file available for a follow-up query.

The correct workflow is question-dependent. Aggregate questions such as 'which test suites failed and how many assertions failed?' are good candidates for code execution. Forensic questions such as 'what was the exact environment variable in the fifth failure?' require either a precise filter that returns the original line or an index/search route. Compactness is helpful only when the computation matches the information need.

6.2 Browser snapshots and network traces

Browser accessibility snapshots and network request lists are frequently verbose because they encode repeated roles, attributes, URLs and layout structure. The project benchmark reports a 56.2 KB Hacker News snapshot becoming 299 bytes, and a 45.1 KB access log becoming 155 bytes. Such reductions are plausible when the agent asks for a page title, a count, a set of links or a filtered status summary. [2]

The danger is that an accessibility snapshot can contain the only clue to a subtle state, and a network trace can include one unusual request that an aggregate summary discards. When debugging a visual or stateful issue, use a two-stage pattern: first ask for a structural summary, then preserve or search the raw snapshot for exact selectors, labels, URLs or ordering evidence.

6.3 Documentation and skills that are queried repeatedly

Documentation is a good fit for `ctx_index` and `ctx_search` when the model needs exact syntax, a code block, a parameter name or a particular section. Indexing avoids repeating the full page in context. Search results can be small while retaining the source text, which is more reliable than asking an abstractive summariser to reproduce an API signature from memory.

The benchmark's documentation results show lower savings than executor summaries: 44% to 93% across the listed examples. That is not a weakness in itself. Exact code and signatures take space. A useful retrieval system should optimise for answerability and correctness, not for the smallest possible snippet. [2]

6.4 Long sessions with compaction

The session layer can help when the agent has made decisions that are not fully represented in the current files: a rejected approach, a constraint from the user, an unresolved blocker or a test that has already been run. Preserving these as structured events reduces the chance of repeating the same investigation after compaction or a restart.

This is most valuable for multi-hour work, but it should be treated as a memory aid rather than an authority. The correct operational pattern is to keep important decisions in durable project artefacts as well: a plan, an issue, a design note or a test. Context Mode's event store should accelerate recovery, not become the only record of truth.

6.5 Privacy-sensitive local processing

If the workflow already has local files, command output or private logs, local execution can reduce the amount of raw content sent to the model provider. This is a real architectural advantage when the compact answer is sufficient. It does not remove the need to inspect the model-visible output for secrets, because the result or an error message can still contain sensitive data.

The privacy benefit should be measured as a data-flow property: which bytes leave the host, which are stored locally, how long they persist and which tools can read them. It should not be described merely as 'private' because the agent itself may still send the compact result, tool definitions, file paths or derived values to a remote model.

7. Where it can be harmful or disappointing

The same mechanism that protects the context window can hide information, add friction or create a new control-plane failure. These are not edge cases to place in a footnote: they define the boundary of safe use.

7.1 Small outputs and unnecessary routing

For a small result, the fixed costs of a hook, subprocess, serialisation, SQLite access or an extra MCP call can exceed the savings. The project benchmark itself includes a 0.4 KB network-request fixture with only 13% reported savings. The current hook code also documents that it cannot know a Bash command's output size before execution, and therefore its default generic nudge is broad. [2, 5]

A community issue reports that the bare Claude Code `mcp__` matcher invokes the hook for every MCP tool, including observe-only tools whose outputs are already small. The issue describes this as overhead with negligible savings and asks for per-tool exclusions or narrower matching. That is a concrete example of a well-intentioned interception policy creating noise. [9]

7.2 Summaries can remove the evidence needed for diagnosis

A summary is a lossy transformation even when it is generated by deterministic code. The model must decide which fields to count, which lines to match and which exceptions to print. A rare failure, a malformed record, a timestamp ordering problem or an unexpected null may disappear. If the task later changes, the discarded detail may be needed, forcing a re-run or an uncertain reconstruction.

The practical safeguard is to distinguish answer extraction from evidence preservation. Store the raw file or index the full result before computing. Return the statistic together with provenance: query, filter, number of rows examined, first and last timestamps, and a pointer or search scope for the raw material. This makes a compact answer auditable.

7.3 Retrieval misses and lexical bias

FTS5 BM25 and trigram search are deterministic and fast, but they do not understand all semantic equivalences. A query for 'authentication failure' may miss a source that says 'credential rejection'. A JSON object may encode a concept in a key or nested value that is not weighted as the model expects. The source uses title weighting, stemming, fuzzy correction and rank fusion to improve recall, but no retrieval layer is perfect.

A particularly dangerous failure is a confident answer based on the top few matching chunks when the relevant exception lies elsewhere. Agents need a retrieval discipline: search with alternative terms, inspect hit counts, ask for absence checks and widen the scope when the result is surprising. The tool should expose enough metadata for the model to know whether it searched five chunks or five thousand.

7.4 The model remains responsible for writing the query or script

Context Mode's strongest executor examples assume that the agent can express the information need as code. That is often easy for counts and filters. It becomes harder for ambiguous natural-language questions, semi-structured files, malformed logs and tasks where the important fact is not known in advance. A syntactically correct script can return a precise answer to the wrong question.

The extra abstraction can also increase interaction length. If the first extraction omits a field, the agent must issue a second call. A native tool result may have allowed the model to notice the field immediately. The right comparison is therefore not 'raw bytes versus compact bytes'; it is 'complete workflow with direct output versus complete workflow with externalised output'.

7.5 Security and prompt injection

The executor turns model-generated code into a local action. A malicious repository file, a poisoned web page or a prompt injection inside an MCP result could influence the model's generated script. If that script reads secrets, deletes files or sends data over the network, Context Mode's local processing capability can increase the blast radius. The project's deny patterns and host-platform permissions reduce but do not eliminate this risk. [1, 4]

The network fetch route deserves similar care. Fetching a URL inside the local process may expose network credentials, internal addressability or a project environment that a remote web fetch would not have had. Restrict egress, keep credentials out of inherited environments where possible, and review the command and code approval text rather than treating an MCP tool name as harmless.

7.6 Runtime, platform and lifecycle failures

The project supports several runtimes and agent hosts, which increases reach but also increases the compatibility surface. A current issue reports a `NODE_OPTIONS` prefix that breaks Bash compound commands in an OpenCode environment. A previous platform issue reported unchanged or increased token usage in VS Code Copilot alongside missing SQLite bindings and failed hook checks. The issue was closed, and the report is version-specific, but it shows why platform claims must be tested rather than assumed. [10, 11]

Long-lived local state has lifecycle costs as well. Orphaned MCP processes, stale SQLite databases, cache growth, symlink behaviour and prompt-cache invalidation have all appeared as operational themes in the current issue tracker. These are not evidence that the design is unsound; they are evidence that the plugin is a substantial runtime component, not a transparent one-line filter.

7.7 Prompt-cache effects are not obvious

Anthropic's current documentation distinguishes context capacity from prompt-cache pricing. Cached prefixes still occupy the context window, while cache reads are priced differently. Cache hits require an exactly matching prefix, including tools, system content and messages up to the cache breakpoint. [12, 13]

Externalising a large result before it enters the prompt may protect the stable prefix, but a hook or injected guidance block can also change the sequence that the platform sees. A third-party report cannot infer the net effect from a local byte counter. Any cost claim must log `cache_creation_input_tokens` and `cache_read_input_tokens` alongside ordinary input and output tokens, and it must compare identical prompts under both configurations.

8. What the evidence actually shows

This chapter separates mechanism evidence from outcome evidence. It is tempting to quote the largest percentage in a README. A more useful approach asks what was measured, against which baseline, with what task and with what independent verification.

8.1 Project benchmark: strong component evidence

The repository's BENCHMARK.md says that its 21 scenarios use captured outputs from real tools rather than synthetic strings. It reports 315 KB of executor inputs becoming 5.5 KB of context, and 60.3 KB of indexed material becoming 11 KB over three searches. It also reports exact code-block preservation for the retrieval scenarios and 125 passing tests across executor, store, MCP integration and ecosystem suites. [2]

Fixture family	Raw	Returned context	Reported reduction	Interpretation
Playwright page snapshot	56.2 KB	299 B	99%	Strong candidate for structural summaries; rare DOM clues may be omitted.
GitHub issues	58.9 KB	1,139 B	98%	Useful for counts and themes; not equivalent to reading every issue.
Access log	45.1 KB	155 B	100%	Excellent for aggregate statistics; weak for forensic ordering unless asked explicitly.
Analytics CSV	85.5 KB	222 B	100%	Good for computed metrics; depends on correct parsing and requested fields.
Documentation search	3.9-33.2 KB	620 B-2,412 B	44-93%	Exact retrieval trades some savings for code and signature fidelity.
Network request list	0.4 KB	349 B	13%	Illustrates the low-value regime and fixed-overhead problem.

The benchmark is valuable because it exposes fixtures and commands. It is not an end-to-end agent benchmark. It does not show whether an agent solved the same tasks, whether the model made more retrieval calls, whether it missed rare errors, or whether the final API invoice was lower. The numbers should be cited as output externalisation measurements, not as universal token-bill savings.

8.2 The benchmark's unit problem

The report uses KB, bytes and a token approximation in different places. A byte reduction is a useful infrastructure measure, but tokenisation is model- and content-dependent. A compact JSON string, a code block and a Unicode-heavy log do not map to tokens in the same way. The benchmark's conversion from 177.1 KB to approximately 45,300 tokens is directionally plausible, yet the methodology is not fully specified in the benchmark file.

For a publishable claim, record the actual provider token usage from the same model and request configuration. Use a token counter only for planning, and treat it as a proxy. Report raw bytes, serialised tool-result bytes, input tokens, cache reads, cache writes, output tokens, number of calls, wall-clock time and task outcomes in separate columns.

8.3 Community adoption is a maintenance signal, not a causal benchmark

The project has a strong community-interest signal: the repository is currently around 20,000 stars with roughly 1,500 forks, and its Hacker News launch discussion reached more than 500 points and more than 100 comments. The repository also shows substantial issue and pull-request activity. These signals increase the probability that real users have encountered compatibility problems and that the code is being maintained. [7, 16]

The same discussion contains substantive criticism. Participants questioned whether pure BM25 was adequate for structured data, whether hooks were too aggressive, whether native Claude truncation already handled some large outputs, and whether external MCP results actually passed through Context Mode. The maintainer's responses clarify design intent, but a discussion is still not a controlled comparison.

8.4 Community reports reveal failure modes

Source	Signal	How to use it
Issue #960	A broad mcp__ PreToolUse matcher can invoke the hook for small observe-only MCP tools.	Treat as evidence of avoidable overhead and a reason to support per-tool exclusions; not evidence of incorrect output.
Issue #567	A VS Code Copilot user reported unchanged or higher token usage with missing SQLite and hook failures on a specific version/platform.	Treat as platform-compatibility evidence; reproduce on the target host before making a general claim.
Issue #1117	A current report describes NODE_OPTIONS injection breaking Bash compound commands in OpenCode.	Treat as a concrete executor integration risk for shell-heavy workflows.
Hacker News	Users reported both useful reductions and concerns about aggressive routing, cache effects, and external MCP bypass.	Use as qualitative evidence and hypothesis generation, not as an effect-size estimate.

8.5 What is not proven as of the research cut-off

- There is no neutral, multi-user, end-to-end study showing a stable percentage reduction in final Claude Code cost across representative coding tasks.
- There is no published independent comparison of task success, omission rate and time-to-correct-answer with native Claude Code, Context Mode, RTK or combinations under identical prompts.
- There is no independent audit showing that session-event recovery improves long-horizon coding outcomes rather than merely producing smaller state snapshots.
- There is no proof that current hooks transform every external MCP result. The current routing source explicitly says the external branch provides guidance without denial or modification. [5]
- Repository stars, claimed organisation logos and launch popularity are useful adoption signals but do not substitute for experimental controls. The README's 'used across teams' statements are project claims, not independently verified customer studies. [1]

9. Relationship to native Claude controls, RTK and Code Mode

9.1 Native Claude context management

Anthropic's current documentation says that system instructions, messages, tool results, images, documents and tool definitions all count towards the context window. It also provides native context awareness, server-side compaction and context-editing strategies such as tool-result clearing. These features mean that Context Mode should be evaluated as a complement to a changing platform baseline, not against an imagined system with no native management. [12]

The distinction is useful. Native compaction acts after a conversation has accumulated and decides what to retain. Context Mode attempts to prevent selected bulky outputs from entering the conversation in the first place, while retaining them in a local store. The two approaches can complement each other, but injected hooks and session guides can also alter the prompt prefix and therefore cache behaviour.

9.2 RTK and deterministic command filtering

RTK-style tools use command-aware filters to reduce the output of common developer commands. Their appeal is predictability: a known command has a known transformation. Context Mode is broader and more programmable. It can process arbitrary files, HTTP responses, browser snapshots, CSVs and documentation, and it can keep the full content searchable. That breadth is useful but gives the model more responsibility for writing the extraction code and gives the runtime more surface area to fail.

The fair comparison is not which tool has the larger headline percentage. Compare per-task success, output fidelity, setup friction, latency and the cost of recovery. A deterministic filter may be preferable for a small set of daily commands, while Context Mode may be preferable for heterogeneous data and exact later retrieval. They can also be layered, but layered hooks need an explicit ownership model so that one tool does not re-process the other's output.

9.3 Code Mode and the input-side problem

Cloudflare's Code Mode is a useful contrast because it addresses MCP tool-definition bloat on the input side. Its server exposes two tools, search and execute, to cover an API with thousands of endpoints, reporting roughly 1,000 tokens for the tool surface. The blog describes server-side execution inside a Dynamic Worker isolate. [14]

Context Mode mainly addresses output-side bloat and session state. It can execute code locally, but it does not reduce the definitions of every external tool already registered in the host, and it does not rewrite external MCP responses. An agent platform may therefore need both strategies: progressive disclosure or Code Mode for tool schemas, and careful externalisation or retrieval for bulky data results.

Layer	Context Mode	Code Mode / progressive disclosure	Native compaction
Primary target	Tool outputs and session state	Tool definitions and API interaction surface	Accumulated conversation history
When it acts	Before selected tools, during local execution, or on later retrieval	Before the model loads or calls detailed API capabilities	After context has accumulated or near a limit
Information retained	Local raw files, indexed chunks, session events	API schema or server-side data, depending on implementation	A model-generated or provider-managed compacted history
Main failure	Wrong extraction, retrieval miss, hook overhead, runtime permissions	Unsafe code execution or incomplete progressive discovery	Summary omission or loss of exact tool-result detail

10. A rigorous evaluation plan for real workflows

The most useful next step for a blog post is not another anecdotal token counter. It is a paired experiment that makes the complete agent configuration the experimental unit: model, reasoning mode, tool definitions, hooks, scaffold, retry policy, runtime and prompt. Changing only the plugin while silently changing any of those other variables weakens the conclusion.

10.1 Experimental arms

Arm	Configuration	Purpose
A: native baseline	Claude Code or target agent with its ordinary context controls and no Context Mode.	Measure the actual current baseline, including native truncation or compaction.
B: Context Mode	Same model, prompts, project, permissions and tasks with Context Mode enabled and documented hook settings.	Measure the total effect of routing, execution, retrieval and session state.
C: selective Context Mode	Enable only selected routes, such as large reads or WebFetch, with small MCP calls allowed through.	Separate the benefit of large-output externalisation from the cost of broad interception.
D: comparison tool	RTK, a deterministic filter or another established solution where applicable.	Test whether the broader Context Mode surface earns its additional complexity.

10.2 Scenario matrix

- Small output: pwd, a version query, a small API response and a small observe-only MCP call. This estimates fixed hook and subprocess overhead.
- Large output: a 100-500 KB log, test run, compiler output and browser accessibility snapshot. Ask both aggregate and forensic questions.
- Exact retrieval: documentation with code examples, API signatures, nested JSON and terminology deliberately varied between the source and the query.
- External MCP: a Slack, issue tracker or database result where the tool is called directly. Measure whether the result is transformed, merely nudged, or ignored by the agent.
- Compaction: a long multi-step task that creates decisions, failed approaches and active files, followed by a forced compaction or restart. Score recovery fidelity.
- Adversarial data: malformed records, one rare error among many successes, prompt injection in a page, a symlinked file and a command containing a shell compound construct.

10.3 Metrics that answer different questions

Metric	Definition	Why it matters
Input tokens	Provider-reported ordinary input tokens per request.	Direct measure of context and cost exposure.
Cache reads and writes	Provider-reported <code>cache_read_input_tokens</code> and <code>cache_creation_input_tokens</code> .	Shows whether a smaller visible result also changes caching economics.
Output tokens	Provider-reported output, including reasoning where applicable.	Separates reduced input from changes in model verbosity or retries.
Tool calls	Count and type of direct, redirected and retrieval calls.	Captures the extra interaction cost of external memory.
Latency	Wall-clock time per tool and end-to-end task.	A compact result is not a win if it makes the workflow materially slower.
Answer accuracy	Exactness against a labelled answer set or test oracle.	Detects summaries and searches that are compact but wrong.
Omission rate	Fraction of planted rare facts, exceptions or exact snippets not surfaced when needed.	Measures the main harm of lossy externalisation.
Pass@1 and repeated success	First-attempt success and success over a fixed retry budget.	Distinguishes reliable improvement from recoverable but noisy behaviour.
Cost per successful task	Total billed cost divided by successful task completions.	The business metric that headline context percentages do not provide.

10.4 Forensic benchmark hygiene

Use fresh or held-out fixtures and record their provenance. Repository-captured outputs are useful for a component test, but an agent can overfit to familiar shapes and the maintainer's preferred queries. If the fixture contains code from public projects, state whether the model may already have seen it during training or prior tool calls. Avoid comparing a carefully engineered Context Mode query with an under-specified baseline prompt.

Run each task multiple times where the model is stochastic, but report both pass@1 and success after a fixed retry budget. Keep the model tier, reasoning setting, temperature or sampling controls, tool schema, permission mode, timeout, working directory and network conditions fixed. Log failures rather than silently rerunning them until they pass.

For retrieval, label every relevant source span and every planted distractor. Ask the same question with synonyms and with exact identifiers. For summaries, plant a rare anomaly and test whether it is surfaced. For session recovery, compare the recovered plan and active-file state with a human-labelled ground truth rather than measuring snapshot bytes alone.

10.5 Minimal reporting table for a blog post

Run	Arm	Task	Input tokens	Cache read	Calls	Latency	Success	Omissions	Cost
01	A	large log aggregate	...	...	...	...	yes/no	...	...
01	B	large log aggregate	...	...	...	...	yes/no	...	...
01	A	rare-error forensic	...	...	...	...	yes/no	...	...
01	B	rare-error forensic	...	...	...	...	yes/no	...	...

11. Practical adoption guidance

A cautious rollout can capture the high-value cases without turning every tool call into a routing experiment.

11.1 Start with the least surprising routes

- Begin with explicit `ctx_execute_file` for large logs, CSVs and test output, where the question is naturally an aggregate.
- Use `ctx_fetch_and_index` plus `ctx_search` for documentation and pages that will be queried more than once or that require exact code snippets.
- Keep a raw artefact, source label, timestamp and query log so that a compact answer can be audited.
- Delay broad Bash and external-MCP guidance until you have measured hook overhead and observed the target agent's behaviour.
- Keep host-level sandboxing and restrictive permissions enabled. Treat arbitrary code execution as a privileged capability, even when the MCP tool looks routine.

11.2 Configure for selectivity

The current routing code exposes configuration for external-MCP nudge cadence and an opt-in minimum Bash-command length. These controls are useful because output size is not observable before a command runs. A team with many small MCP observations should consider an allowlist or exclusions for lightweight tools while reserving aggressive routing for known large-output sources. The open issue requesting per-tool exclusion is a reasonable operational requirement, not an academic preference. [5, 9]

Avoid treating the default as universally optimal. The default is designed to protect against surprise floods. A mature setup should tune it using observed distributions: percentage of calls over a threshold, bytes avoided, hook latency, false redirects and agent recovery calls. Configuration should be checked into project documentation so that experiments remain reproducible.

11.3 Use a two-stage answer contract

For large structured data, ask the executor to return both the answer and an audit envelope: source, rows or lines examined, filters, counts, min/max timestamps and a pointer to the raw or indexed material. This keeps the response compact without pretending that the summary is the evidence itself.

For indexed material, ask the model to report query terms, hit count, source scope and whether it searched for counterexamples. A search result should be treated like a database query, not like the complete document. This practice makes retrieval misses visible in both human review and automated evaluation.

11.4 Keep the blog claim narrow

A defensible headline would be: Context Mode can keep large, queryable tool outputs out of the active model context, with substantial fixture-level reductions, but its end-to-end cost and quality effects depend on routing, retrieval and task type. A headline such as 'cuts Claude Code token bills by 98%' would overstate what the available evidence demonstrates.

12. Balanced verdict

Context Mode is more than a marketing wrapper around truncation. Its code shows a coherent architecture: a hook-controlled routing layer, a local executor, output limits, FTS5 indexing, lexical and substring retrieval, caching and a structured session-event store. The project benchmark is transparent enough to be useful, and the community is large enough that the implementation has received meaningful real-world scrutiny. [1, 2, 3, 4, 5, 7]

The most valuable contribution is the separation between storage and working context. Large logs, snapshots and documents can remain available without becoming permanent conversational ballast. Exact retrieval is a stronger design than simply asking a model to summarise everything, because code snippets and signatures can be recovered. Session continuity also addresses a real long-horizon problem.

The limits are equally real. Summaries can omit rare evidence; lexical retrieval can miss semantic matches; the model must write correct extraction code; hooks can over-intercept small calls; arbitrary local execution expands the security surface; runtime differences can invalidate the promised behaviour; and external MCP output is not currently transformed by the hook. The current issue tracker is not a footnote to the design: it is evidence of the operational complexity introduced by the system. [9, 10, 11]

For a senior engineering or data-science workflow, the appropriate stance is therefore promising, useful and not yet proven as a universal cost-saving layer. Adopt it first where the information need is naturally aggregate or searchable, instrument the entire task, and compare against native controls and deterministic filters. If the A/B study shows lower cost per successful task without a meaningful omission or latency penalty, then the result will be strong enough to support a blog post. If it does not, that negative result will also be valuable because it will identify where output externalisation is noise rather than leverage.

Appendix A. Glossary

Term	Meaning in this paper
Context window	The model's current working memory: system content, messages, tool definitions, tool results and generated output that count for the request.
Externalisation	Keeping raw data outside the model-visible conversation and returning a compact result or pointer.
MCP	Model Context Protocol, a tool interface through which an agent discovers and calls external capabilities.
Hook	A host callback before or after a tool call. In Claude Code, PreToolUse can add context, modify or deny a call.
FTS5	SQLite's full-text search extension, used here for local lexical indexing and retrieval.
BM25	A term-frequency and inverse-document-frequency ranking function used by FTS5; it is lexical, not semantic.
RRF	Reciprocal Rank Fusion, a method for combining multiple ranked result lists.
Prompt cache	A provider-side cache for identical prompt prefixes. A cache hit can reduce price but the cached prefix still occupies context.
Task success	Whether the complete agent workflow produces the correct result, not whether one tool response is small.
Omission error	A relevant fact present in the raw source but not surfaced by the compact output or retrieval path.

Appendix B. Source register and evidence grading

The links below are preserved so that a future blog post can navigate from each claim to the original material. URLs were checked during the research cut-off stated on the cover. Repository pages and issue reports are mutable; record the commit, version and access date when publishing a final article.

[1] [Context Mode README](#). Primary project documentation: tools, hooks, runtimes, security notes, self-reported benchmark summary, caching and session continuity.

[2] [Context Mode BENCHMARK.md](#). Project-run fixture benchmark: 21 scenarios, raw/context sizes, exact-code claim, test counts and reproduction commands.

[3] [ContentStore source](#). Primary implementation evidence for SQLite schema, FTS5, BM25 weighting, chunking, trigrams and retrieval helpers.

[4] [Executor source](#). Primary implementation evidence for child processes, stdout/stderr capture, caps, timeouts and environment handling.

[5] [Routing source](#). Primary implementation evidence for redirects, guidance, external-MCP detection and the explicit no-deny/no-modify branch.

[6] [Claude Code hook adapter](#). Primary implementation evidence for current Claude Code matcher coverage.

[7] [Hacker News discussion](#). Community evidence: adoption interest, external-MCP debate, BM25 critique, hook overhead concerns and cache discussion.

[8] [Context Mode v1 release post](#). Maintainer account of release scope, adoption and self-reported improvements; treated as project evidence, not independent validation.

[9] [Issue #960: broad mcp matcher](#). Community-reported overhead from routing small observe-only MCP calls; open issue at the research cut-off.

[10] [Issue #567: VS Code Copilot token usage](#). Version-specific community report of unchanged or higher usage with missing SQLite and hook failures; closed issue.

[11] [Issue #1117: NODE_OPTIONS and Bash compound commands](#). Current community report of a shell integration failure in an OpenCode environment.

[12] [Anthropic context windows documentation](#). Primary provider documentation for context composition, context awareness, compaction, context editing and window limits.

[13] [Anthropic prompt caching documentation](#). Primary provider documentation for exact prefix matching, cache pricing and cache usage fields.

[14] [Cloudflare Code Mode](#). Primary vendor architecture comparison for server-side code mode, tool-definition reduction and sandboxed execution.

[15] [SQLite FTS5 documentation](#). Primary database documentation for FTS5 querying and BM25 ranking concepts.

[16] [Context Mode GitHub repository](#). Current repository status and adoption signal; stars and forks are treated as interest and maintenance signals only.

Evidence map

Evidence type	Examples in this paper	What it can support	What it cannot support
Repository fact	Source code, README, current routing comments	How the current system is designed and what it claims to do.	Whether every platform behaves identically in production.
Project benchmark	BENCHMARK.md fixture reductions and tests	Component-level output and retrieval behaviour on disclosed fixtures.	Universal cost reduction, task success or unbiased comparison.
Community signal	Hacker News, GitHub issues and usage reports	Real failure modes, adoption interest and hypotheses for testing.	A controlled effect size or population-wide conclusion.
External primary evidence	Anthropic and Cloudflare documentation	Context mechanics, caching, native controls and a contrasting input-side design.	The specific effectiveness of Context Mode.
Independent validation	Not found for end-to-end Context Mode cost and quality	This absence is itself an evidence boundary.	It does not prove the tool is ineffective; it limits the strength of public claims.

Suggested next experiment. Run the six-scenario A/B matrix in Chapter 10 on the exact agent, model, prompt and project that will appear in the later blog post. Preserve raw fixtures, usage JSON, hook logs, retrieval queries and failed runs. Publish the whole workflow, not only the best reduction percentage.